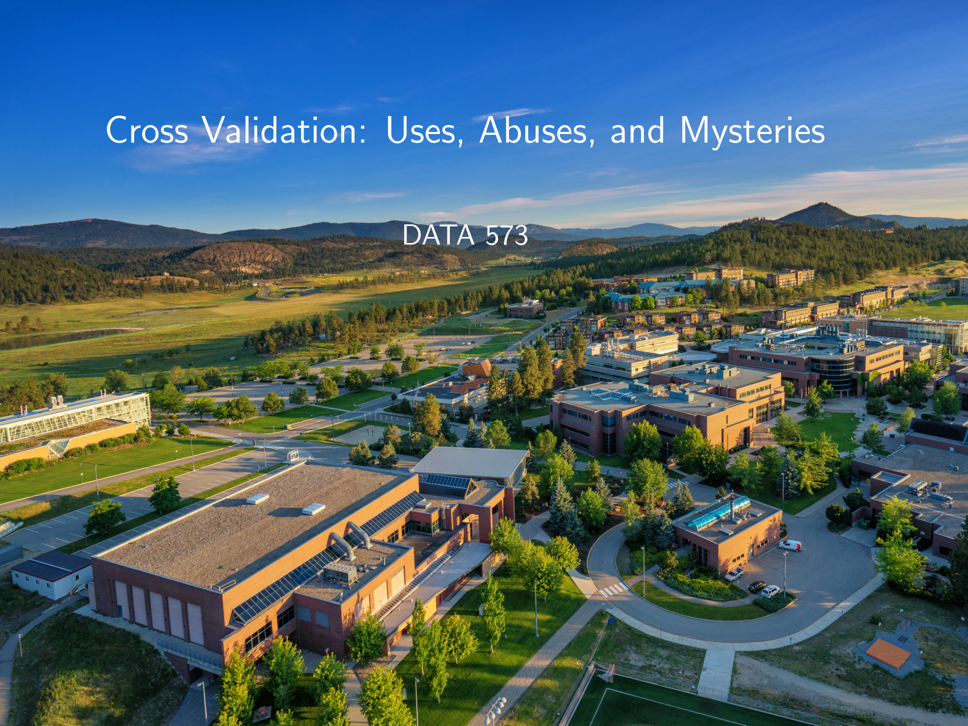
Cross Validation: Uses, Abuses, and Mysteries

DATA 573



Loss

- To fit a model, we need a function to optimize.
- This is commonly referred to as loss, and defines how we penalize errors in prediction.

$$L(Y, f(X))$$

- For continuous response, squared loss is by far the most common.

$$L(Y, f(X)) = (Y - f(X))^2$$

- Why? Because it's usually easiest — smooth, differentiable, etc

Why not?

- Squared loss is not universally sensible...

	Mod1	Mod2
$y_1 = 10$	0	9
$y_2 = 40$	50	20
$y_3 = 100$	90	101
Sum Sq Error	300	402
Sum Abs Error	30	22

- The type of error you'd *like* to minimize is generally problem specific, though you may be constrained by what is feasible from an optimization standpoint.

Splitting Up the Sample

- We've already seen training and testing sets:

Full Sample Index	
$1, 2, 3, 4, 5, \ldots, 15, 16, 17, \ldots, 198, 199, 200$	

Randomly Sample

$\swarrow \quad \searrow$

Training Index	Testing Index
$2, 3, \ldots, 7, 9, \ldots, 199$	$1, 4, 5 \ldots, 8, \ldots, 198, 200$

- In this case, we fit the model on the training set, and estimate the MSE using the testing set.

- Of course, in data-poor environments, this can be costly from biased estimate standpoint.

Leave One Out Cross-Validation

	Full Sample Index	
	$1, 2, 3, 4, 5, \ldots, 15, 16, 17, \ldots, 198, 199, 200$	
	Systematically divides into...	
CV Set	Training Index	Testing Index
1	$2, 3, \ldots, 200$	1
2	$1, 3, \ldots, 200$	2
3	$1, 2, \ldots, 200$	3
$\vdots$	$\vdots$	$\vdots$
200	$1, 2, 3, \ldots, 199$	200

Leave One Out Cross-Validation

- We then fit our model to each of the $i = 1, \ldots, n$ CV training sets, and receive a prediction for the i^{th} testing (or left out) observation.

- If we define $MSE_i = (y_i - \hat{y}_i)^2$ where $\hat{y}_i$ is found from the i^{th} model

- We can then estimate the MSE of the model using

$$CV_{(n)} = \frac{1}{n}\sum_{i=1}^{n} MSE_i$$

LOOCV

- Pros
  - Approximately unbiased estimation of the extra-sample error (the long-run error if we train on a new sample of the same size pulled from the same population)
  - Deterministic estimate if the model is also deterministic
- Cons
  - Computationally expensive for large sample sizes
  - Relatively high variance estimation of the extra-sample error

k-Fold CV

- We can randomly subdivide the sample into k approximately equally-sized and non-overlapping sets.
- Each set can be considered a validation set, with the remainder of the data used to train the model.
- Then we can calculate the MSE for each validation set j

$$MSE_j = \frac{1}{\sum_{i=1}^{n} I(i \in j)} \sum_{i \in j} (y_i - \hat{y}_i)^2$$

- And estimate the test MSE with

$$CV_{(k)} = \frac{1}{k} \sum_{j=1}^{k} MSE_j$$

k-Fold Cross-Validation

	Full Sample Index	
	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20	
	Randomly divides into...	
CV Set	Training Index	Testing Index
1	1, 2, 4, 5, 6, 8, 9, 11, 13, 14, 15, 16, 17, 18, 19, 20	3, 7, 10, 12
2	1, 3, 5, 6, 7, 8, 9, 10, 11, 12, 13, 15, 16, 17, 18, 20	2, 4, 14, 19
3	1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 14, 15, 17, 19, 20	5, 13, 16, 18
4	1, 2, 3, 4, 5, 7, 10, 11, 12, 13, 14, 16, 17, 18, 19, 20	6, 8, 9, 15
5	2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 13, 14, 15, 16, 18, 19	1, 11, 17, 20

k-Fold CV

- Assume a relatively small k, common choices are $k = 5, 10$.
- Pros
  - Lower variance than LOOCV for estimating extra-sample error — generally minimizes bias-variance tradeoff for this estimation
  - Computational costs are controlled by choice of k
- Cons
  - Additional bias in estimation vs LOOCV — tends to slightly overestimate due to smaller training sample sizes
  - Non-deterministic estimate even for deterministic model-fits

General Setup

- Modelling in data-rich environments should always consist of Training, Validation, and Testing sets.
- Training - for estimating the model
- Validation - for tuning the model
- Testing - for checking model performance
- 'Good' models should show similar performance across all three sets. Large discrepancies in performance is indicative of overfitting.

Cross-Validation: A deeper look

- What is cross-validation actually estimating?
- ESL (Ch 7) goes deeper than ISLR, and a recent article in JASA[1] sheds further light specifically in the linear model case.
- General claim: given some training set τ, we would be generally interested in the conditional expectation

$$\text{Err}_{\tau} = \mathbb{E}_{X_0,Y_0}[L(Y_0, \hat{f}(X_0)) \mid \tau]$$

- In words, basically how *this model I'm specifically looking at* would be expected to perform for future data
- This expectation is surprisingly difficult to estimate!

[1] Bates, Hastie & Tibshirani (2023) 'Cross-validation: what does it estimate and how well does it do it?', *Journal of the American Statistical Association*

Cross-Validation: A deeper look

- Cross-validation (and similar computational approaches, like bootstrap OOB error) don't estimate Err_τ well!

- Instead, they effectively estimate

$$\text{Err} = \mathbb{E}_\tau \, \mathbb{E}_{X_0,Y_0}[L(Y_0, \hat{f}(X_0)) \mid \tau]$$

- In words, basically how *this model fitting process* would be expected to perform given a new training set (of the same size) for future data

- Aside: as a methodological researcher, my interests generally lie in the more general 'Err' anyway

- But from a practitioners standpoint, in data-poor environments, we generally struggle to estimate how THE fitted model will behave in the long run!

Cautionary Tales

- Let's jump to R for some more cautionary tales surrounding training/testing and CV...

Foreshadowing...

- It was generating this lecture up to this point several years ago that I realized ESL had a major flaw in it...
- We'll get there...

What is CV typically used for?

- CV use cases are varied and, as we'll argue, often abused.
- Loss estimation: as already discussed, *just* estimate Err for information's sake
- Model-selection: given $\hat{f}_k$ for $k = 1, \ldots, K$, choose k such that Err_k is minimized.
- Hyper-parameter tuning: $\hat{f}_k$ may have several, related forms which the researcher has no *a priori* argument to set, so these are tested and selected via CV
- Inference on parameter estimators: see jacknife for computing bias and standard errors

What is CV used for?

- The mere usage of CV in no way guarantees sensible modeling has taken place...
- In modern data analysis pipelines, particular care needs to be taken.
- Good rules of thumb:
  - CV should only be used for ONE task — you cannot use the same folding process for both hyperparameter tuning and realistic Err estimation. (See 'nested CV')
  - All data processing steps (with very few caveats) should be performed within the CV folds.

Terms: Supervised vs Unsupervised 'Processing'

- 'Supervised' methods are any sort of sample-dependent change that is applied to X that uses Y
  - Filtering X by its relationship to Y ($\text{Cor}(X, Y)$ as we just saw)
  - Transforming X by its relationship to Y (PLS)
  - Adding predictors to X via some knowledge of Y (clustering on Y)

- 'Unsupervised' methods are any sort of sample-dependent (or stochastic) change that is applied to X (that never sees Y)
  - Filtering X via patterns in X (max variance)
  - Transforming X via patterns in X (normalization)
  - Adding predictors to X via patterns in X (clustering on X)
  - Doing something 'randomly' to X (toy example to follow later)

Recent Work

- ESL correctly suggests that you CANNOT use supervised techniques prior to entering CV (and expect sensible results)
- ESL INCORRECTLY suggests that you can use unsupervised techniques prior to entering CV
- Moscovich and Rosset[2] prove that unsupervised methods on the full training data also introduce bias to CV Err estimation.

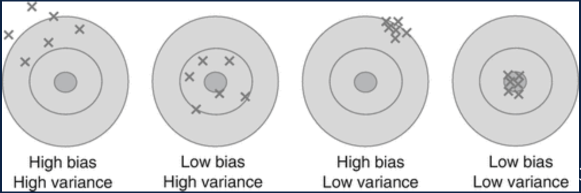
[2] Moscovich & Rosset (2022). On the cross-validation bias due to unsupervised preprocessing. *Journal of the Royal Statistical Society Series B: Statistical Methodology*.

In other words

- Any sample-dependent pre-processing done to the full data is biasing your next steps that use CV (or similar computational tricks)
- This undoubtedly has large ramifications throughout the scientific literature, especially in areas that deal with limited sample sizes (bioinformatics, pilot studies, etc)
- There are software packages that contain this fundamental flaw(!!!)
- But leaving this concern aside...

Statistical Learning

- We generally discuss (in both estimation and modelling) the primary concerns of 'bias' and 'variance'. Estimation...

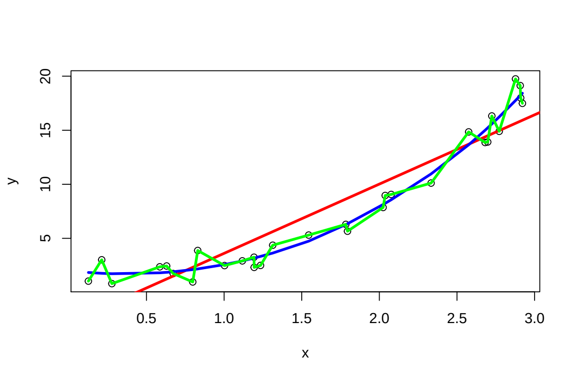


[3]

[3] (2011). Bias Variance Decomposition. In: Sammut, C., Webb, G.I. (eds) Encyclopedia of Machine Learning. Springer, Boston, MA.

Statistical Learning

- Modelling...



Mysteries...

- So Moscovich & Rosset prove a bias in Err using unsupervised processing...
- What about the variance??
- Much of my sabbatical grant proposals surrounded this question in some form. Nobody has published on this (to my knowledge) and there are still many open questions. Let's look at a toy example...

Coin Flip Filtering

- Consider a simple example with two (independent) predictors X_1, X_2 and one response Y.

- Suppose we adopt a modeling process wherein we linearly model

$$Y = \beta_0 + \beta_1 X_S$$

  where $S = 1$ if a coin flip results in heads, and $S = 2$ if a coin flip results in tails.

- Computing the conditional expected loss for a fixed training set $\tau = (\tilde{X}_1, \tilde{X}_2, \tilde{Y})$, where $\tau_S = (\tilde{X}_S, \tilde{Y})$ then becomes

$$\begin{aligned} \text{Err}_\tau &= \mathbb{E}_S[\text{Err}_{\tau,S}] \\ &= 0.5[\mathbb{E}_{X_1,Y}[L(Y, \hat{f}(X_1)) \mid \tau_1] \quad + \\ &\quad 0.5[\mathbb{E}_{X_2,Y}[L(Y, \hat{f}(X_2)) \mid \tau_2]. \end{aligned}$$

Coin Flip Filtering

- We'll consider three illustrative scenarios.
- Scenario 1: both X_1 and X_2 are equally useful for linearly modeling Y
- Scenario 2: neither X_1 nor X_2 are useful for linearly modeling Y
- Scenario 3: only X_1 is useful for linearly modeling Y

Coin Flip Filtering

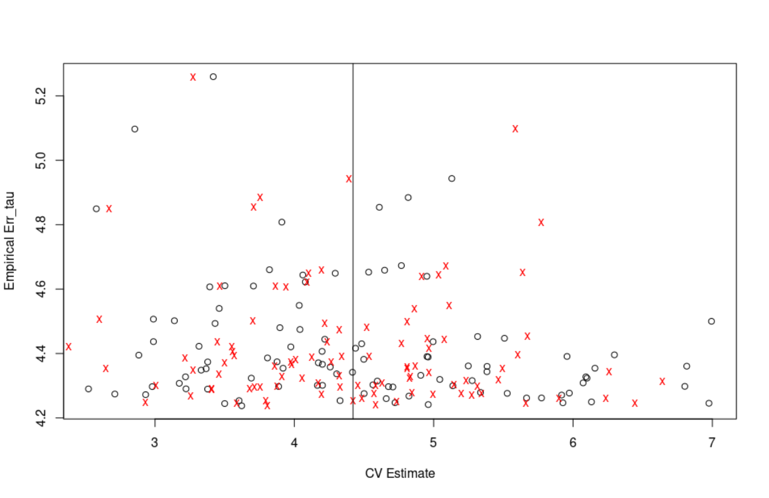
- In Scenario 1 (equally useful), and Scenario 2 (neither useful)

$$\mathbb{E}_{X_1,Y}[L(Y,\hat{f}(X_1)) \mid \tau_1] \approx \mathbb{E}_{X_2,Y}[L(Y,\hat{f}(X_2)) \mid \tau_2]$$

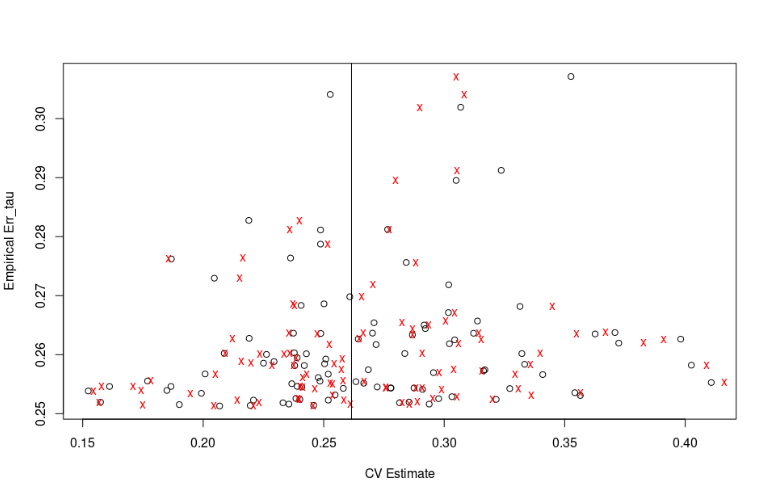
- This suggests that filtering conditional on the full training data τ (aka, one coin flip) should have negligible effects on the bias and variance of our Err estimation.

- We simulate 100 replications ($n_\tau = 50$ in each case), computing empirical Err_τ from a large test set ($n_{\text{test}} = 50,000$). Averaging Err_τ across the 100 replications provides an empirical Err.

Equally Useful



Neither Useful



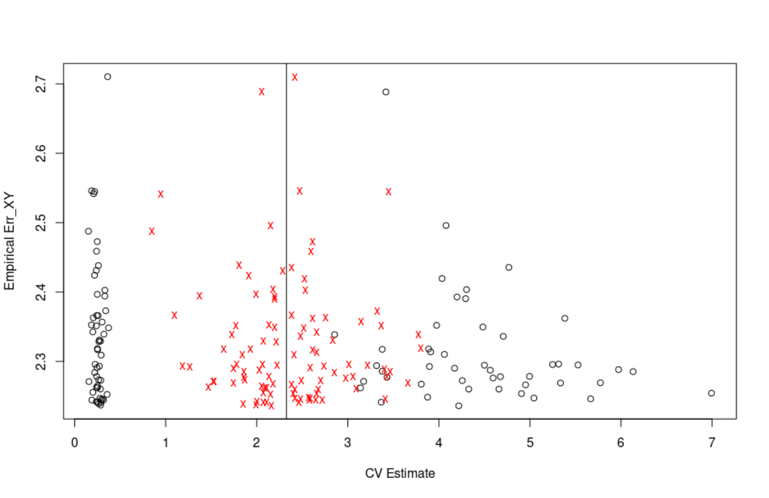
Coin Flip Filtering

- Scenario 3 (only X_1 useful) suggests

$$\mathbb{E}_{X_1,Y}[L(Y,\hat{f}(X_1)) \mid \tau_1] \ll \mathbb{E}_{X_2,Y}[L(Y,\hat{f}(X_2)) \mid \tau_2]$$

- So filtering conditional on the full training data τ (aka, one coin flip) should have drastic effects on the bias and variance of our Err estimation, versus flipping the coin within each CV fold.

X_1 Useful



Summary

- First two scenarios are silly. You need to pre-assume that whatever you're doing to X won't affect your loss function, regardless of the outcome.
- There is no data analyst that is applying some procedure to their data under the assumption that it will have no affect on their modeling results. It's safe to assume they think it will improve their loss...
- The third example, while contrived for mathematical convenience, is relevant. We see an improper workflow giving a bias (either overly optimistic or overly pessimistic) AND increased variance in estimating the target.

'Towards Realistic' Toy Example

- A common screening approach in certain 'high p, low n' settings is to consider the variables with largest variance and build a model with them
- For simplicity, and tractability, let's slightly adjust our toy example...

Max Variance Filtering

- Two (independent) predictors X_1, X_2 and one response Y.
- Suppose we adopt a modeling process wherein we linearly model

$$Y = \beta_0 + \beta_1 X_M$$

  where $M = 1$ if $s_1^2 > s_2^2$, $M = 2$ otherwise.
- We'll assume that X_1 is the only useful predictor
- Importantly, we'll generate data such that $\sigma_1^2 = 1.1$ and $\sigma_2^2 = 1$
- In other words, our screening process is somewhat sensible. The truly higher variance predictor SHOULD be selected.

Max Variance Filtering

- Conveniently, we can easily work out via the F-distribution

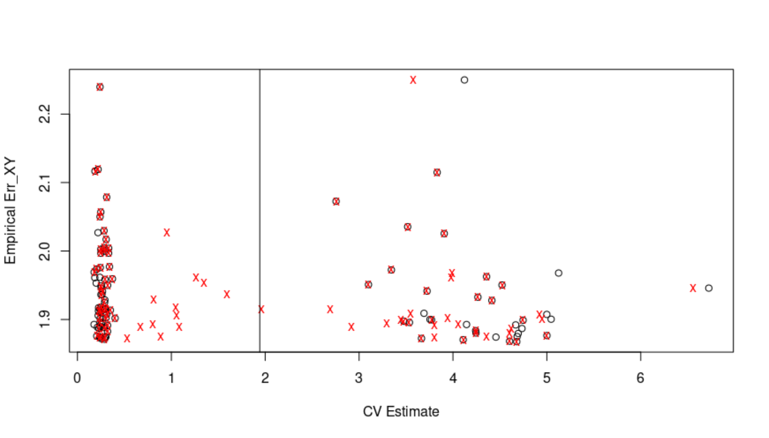
$$P\left(\frac{\sigma_1^2}{\sigma_2^2}\frac{s_2^2}{s_1^2} < 1.1\right) \approx .63$$

$$P\left(\frac{s_2^2}{s_1^2} < 1\right) \approx .63$$

- Now we can expand the conditional expected loss again

$$\begin{aligned}\mathrm{Err}_\tau &= \mathbb{E}_M[\mathrm{Err}_{\tau,M}] \\ &= 0.63[\mathbb{E}_{X_1,Y}[L(Y,\hat{f}(X_1)) \mid \tau_1] \quad + \\ &\quad 0.37[\mathbb{E}_{X_2,Y}[L(Y,\hat{f}(X_2)) \mid \tau_2].\end{aligned}$$

Max Variance Screening



Takeaway

- In short, safest to:
  - Only use CV for one task (or consider nested CV for multi-task)
  - Incorporate all processing tasks within the CV folds
- Doing any processing, even the seemingly mundane, on the full training data generally renders whatever you're doing with CV flawed!
- 'How flawed?' — depends on training size, the process used, etc. Overall, this theme is a topic of ongoing research :)


UBC
THE UNIVERSITY OF BRITISH COLUMBIA